Ray AI Platform DevOps contract submission

Prepared by Sumit Sharma for the DevOps and Platform Engineering contract role evaluation.

This submission presents a tested implementation and the operational design needed to take it into production. The production recommendation is a private multi-AZ platform with recoverable batch execution and a separately tested regional recovery path. High availability and disaster recovery below are proposals, not deployed capabilities or measured service guarantees.

The implementation runs a corporate data pipeline on Kubernetes using Ray Data, a dedicated EntityResolver actor, transactional Apache Iceberg updates and CPU revenue-model training. The repository includes separate Terraform state layers, EKS and KubeRay, scoped workload identity, OIDC delivery, monitoring and centralized AWS logs. Evidence distinguishes completed cloud tests, the local rehearsal and prepared changes that have not yet been validated in AWS.

Repository: https://github.com/SumitSharma-DevOps-3107/contexq-assignment Review branch: codex/security-hardening Pull request: https://github.com/SumitSharma-DevOps-3107/contexq-assignment/pull/2

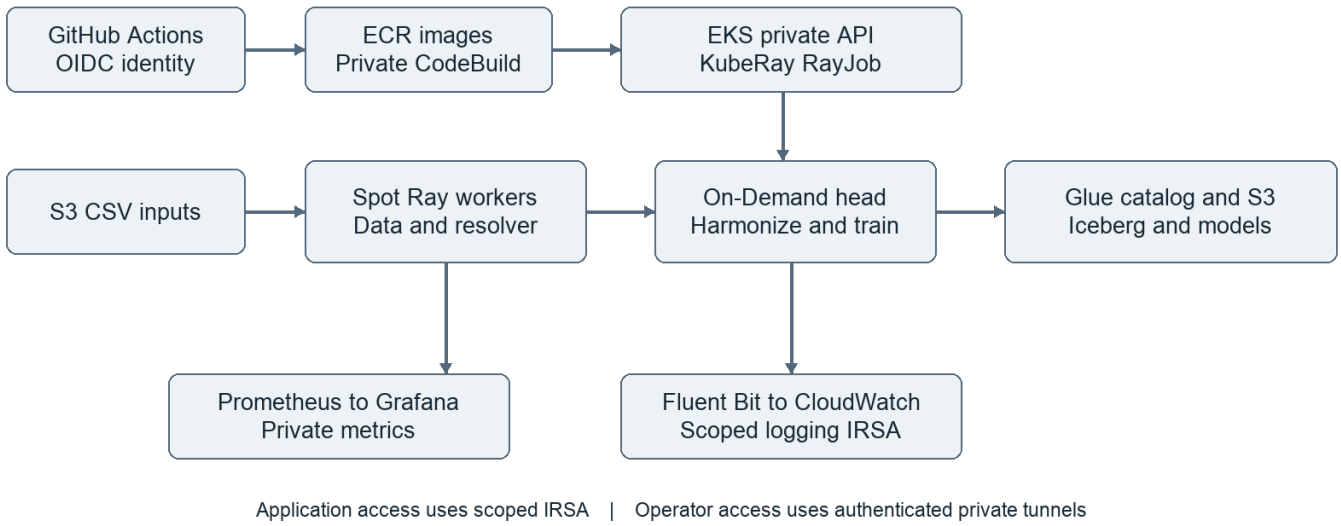
Requirement review

The assignment permits amendments to its skeleton. The executable is python -m ray_platform.pipeline; the implementation replaces the simulated LLM/hash resolver with deterministic identity keys. No LLM credential or GPU is needed for this synthetic fixture. It does not claim real-world fuzzy matching.

Assignment requirement	Implementation and evidence	Qualification
Two CSV datasets with at least 1000 rows each	datasets.py creates 1200 rows per source; 2400 input rows produce 1800 companies	Synthetic fixture
Resolve entities and transactionally update Iceberg	SHA-256 identity from country and normalized registration ID; real catalog commit and repeat/update tests	Single writer; not a distributed join
Revenue prediction from harmonized data	Training reads committed table; RandomForest uses 1440 training and 360 held-out companies	Synthetic accuracy; CPU driver training
Decoupled Terraform layers and managed Kubernetes	bootstrap, data, compute, platform and ci states; EKS module	Private access required for platform
Buckets and Iceberg catalog	S3 and Glue; cloud-validation-summary.json	Existing versions require separate encryption migration
KubeRay and separate node pools	On-Demand system pool and autoscaled Spot worker pool	One system node is not HA
Spot cost and resilience strategy	FIS interruption terminated one worker; actor recovered elsewhere; idempotent rerun	Head failure and distributed training checkpointing not tested

Assignment requirement	Implementation and evidence	Qualification
Ray Data and Train configuration explanation	README and scaling-and-alternatives.md cover resources, object stores, shared memory and distributed trainer configuration	Distributed Ray Train is design-only
EntityResolver placement	Ray custom resource plus pool labels and worker taint; placement evidence	Worker capacity required
Workload identity and access boundaries	Driver/actor IRSA verified; state-bucket access denied	Least privilege demonstrated, not a complete zero-trust certification
PR Terraform plan	OIDC and private CodeBuild; successful historic PR plan	Cloud CI deliberately paused between demo windows
Merge image build and RayJob execution	Successful recorded merged release, ECR digest and RayJob cleanup	New golden-image gate still blocks vulnerable candidates
Prometheus metrics	Head/worker, task, CPU and active-cluster families verified	Ephemeral short retention
Secure Ray Dashboard	Loopback forwarding and token authentication; unauthenticated request returned 401	No public ingress
Centralized head and worker logs	Fluent Bit and scoped IRSA to CloudWatch; observability-validation.json	Ray UI capture shows driver logs; separate CloudWatch API evidence
Repository, setup, diagram and serverless alternative	README, infra documentation, diagram and managed alternative below	Final review branch must be included in handoff

Architecture



Cloud data path: S3 CSV inputs → Ray Data workers → EntityResolver actor → driver harmonization → Iceberg table in S3 with Glue catalog → model and metrics in S3. The head advertises zero task CPUs; the

actor requests the entity_resolver custom resource available only on the worker group. Kubernetes labels and taints enforce physical separation. The local equivalent uses a SQLite catalog and filesystem warehouse solely for reproducibility.

Delivery path: GitHub Actions → repository-scoped OIDC → ECR image build and private CodeBuild → EKS RayJob. PR plans do not apply infrastructure. The platform layer owns KubeRay, autoscaling and collectors; application pods use scoped IRSA, separate from the provisioning role. Prometheus feeds Grafana; Fluent Bit sends head and worker stdout/stderr to CloudWatch. Dashboards remain private.

Local setup

Prerequisites: Docker Desktop, Python 3.11, kind, kubectl, Helm, make and Git. The tested ARM64 Mac has 16 GiB RAM; Docker is allocated 6 CPUs and about 8 GiB. Use synthetic data only. A fresh machine needs internet access for pinned tools, images and Python dependencies.

```
git clone https://github.com/SumitSharma-DevOps-3107/contexq-assignment.git
cd contexq-assignment
git checkout codex/security-hardening
bash scripts/install-kubectl.sh
export PATH="$PWD/.tools:$PATH"
make preflight
make setup
make test
make run
make image-offline
make local-up
make submit
make status
```

Inspect RayJob status and logs using the explicit kind-ray-assignment context. Outputs persist under runtime/kubernetes across Ray cluster lifecycles. Submit a second job after completion: inserts and updates should both be zero and the Iceberg snapshot should remain unchanged. make local-down removes only the named kind cluster and preserves outputs.

For Grafana run bash scripts/grafana-local.sh; use localhost port 3000 and the private runtime password file. Never publish credentials or raw runtime files. The screenshot rehearsal retained its Ray cluster temporarily by removing TTL when shutdownAfterJobFinishes was false, then explicitly removed both test jobs. Normal jobs keep automatic cleanup. Run bash scripts/test-monitoring-alerts.sh to validate alert pending, firing and recovery against deterministic fixtures.

AWS setup and teardown

Use Singapore ap-southeast-1 with the authorized assignment profile. Authentication is browser login followed by role assumption; no static keys are stored in Git. The reproducible private-access and screenshot procedure is in docs/eks-walkthrough.md. Source scripts/aws-env.sh, verify account identity and gross spend, and generate ignored configuration with scripts/aws-configure-project.py. Set an explicit bounded lease; configuration generation does not silently extend an old deadline.

Apply reviewed plans in dependency order: bootstrap, data, compute, then platform from private connectivity. Initialize each remote layer with its backend.hcl. The ci layer requires compute and data outputs. Current source uses private-only EKS access and customer-managed encryption keys. The earlier /32 public API setup is historical and must not be copied into a new deployment. Use an authenticated private path and retain TLS certificate verification.

Upload generated CSVs to the private data bucket input prefix. Use an immutable ECR digest with scripts/render-aws-job.py and submit through the private API. Verify SUCCEEDED, S3 artifacts, Glue table metadata, workload identity, the denied state-bucket read, and repeat-run idempotence. Confirm Prometheus targets and CloudWatch streams for both node types. Do not enable cloud CI until its settings, network path, image policy, budget and lease checks all pass.

Destroy platform while the API is reachable, followed by compute and any dependent CI networking resources in the documented order. Confirm node groups, cluster, NAT and EIP are gone. Retain state, evidence and storage keys while encrypted data or versions need them. The lease guard is a fallback; delayed billing and async cleanup mean it is not a hard dollar cap.

Module and delivery design

infra/modules/eks contains separate variables, locals, outputs and version constraints, validated node-pool inputs, dynamic worker taints and instance/volume tags. The stable system/worker resource addresses avoid needless state moves. The root pins provider versions and lock files; the reusable module declares a compatible version range. Its VERSION and CHANGELOG describe 1.0.0-rc.1; no remote release tag is claimed. Pin a reviewed commit for Git consumption.

Security checks include source and resolved-plan scanning, secret scanning, conventional PR titles, local hooks and required CI. Peer approval is temporarily paused at the owner's request; restoration policy is retained. The public GitHub Free repository contains sanitized evidence only. Atlantis is a plan-only configuration, not a live webhook service. These distinctions are intentional.

EKS validation

On 26 October 2026, the refactored module provisioned EKS 1.34 in Singapore with a private-only API and KMS secrets encryption. Both managed node pools became Ready. Operator access used Systems Manager and a loopback CONNECT bridge with the original hostname and CA verification. No ALB, domain or public ingress was created. The bounded lease expires at 09:19:05 UTC that day.

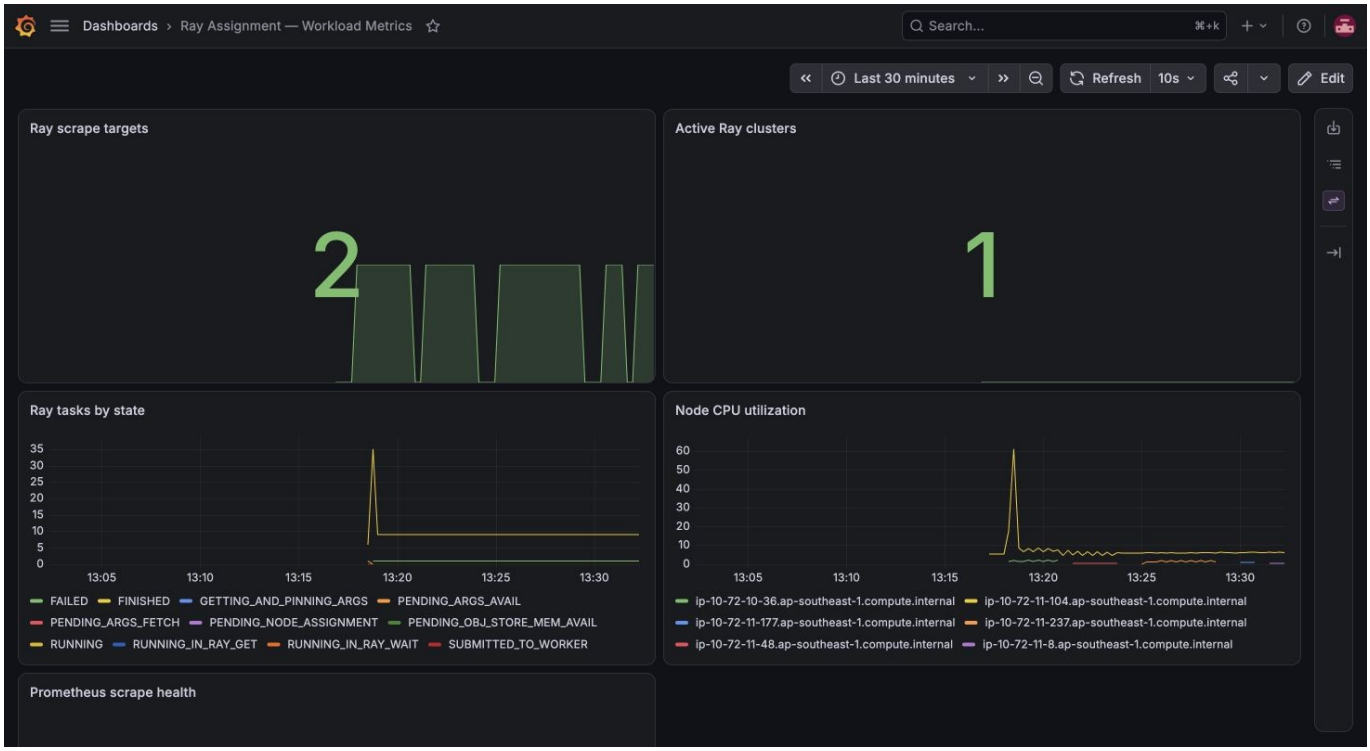
RayJob eks-screenshots-fz4wk succeeded. Its Ray submission eks-screenshots-fz4wk-wbpbn completed at 07:48:26 UTC in 15.072 seconds. The pipeline read 2400 S3 input rows and produced 1800 companies against the existing Glue/Iceberg table, with zero inserts and zero updates on this repeat run. Driver and worker identities both used the scoped pipeline IRSA role. Prometheus verified two healthy Ray targets. Fresh CloudWatch events from both ray-head and ray-worker were collected for this run. Unauthenticated Grafana and Ray API requests both returned HTTP 401. Alert fixture tests passed inside EKS.

The run used the previously tested immutable ECR digest ending in 0c16a242adeb4. This was a manual evidence execution, not golden-image promotion or a rerun of paused cloud CI. Earlier checks verified SSE-KMS state writes and data-bucket defaults. Existing object versions and CI artifact encryption were not migrated in this run. Machine-readable evidence is in docs/evidence/eks-rehearsal/final-capture-run.json.

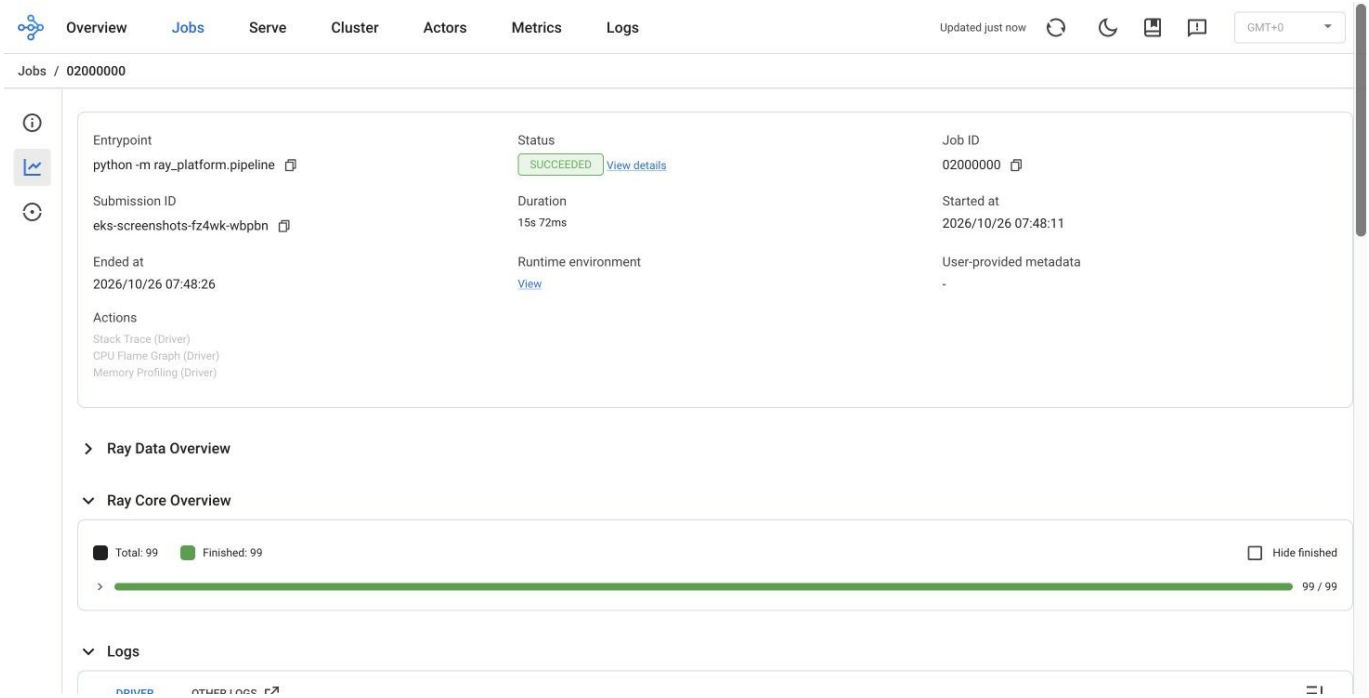
At 13:34 UTC on 26 October, AWS confirmed the EKS cluster absent, with no assignment EC2 instances, active NAT gateways or allocated Elastic IPs in Singapore. Retained storage, encryption keys and CloudWatch logs remain. Out-of-band compute cleanup requires Terraform refresh and reconciliation before the next deployment.

Demonstration evidence

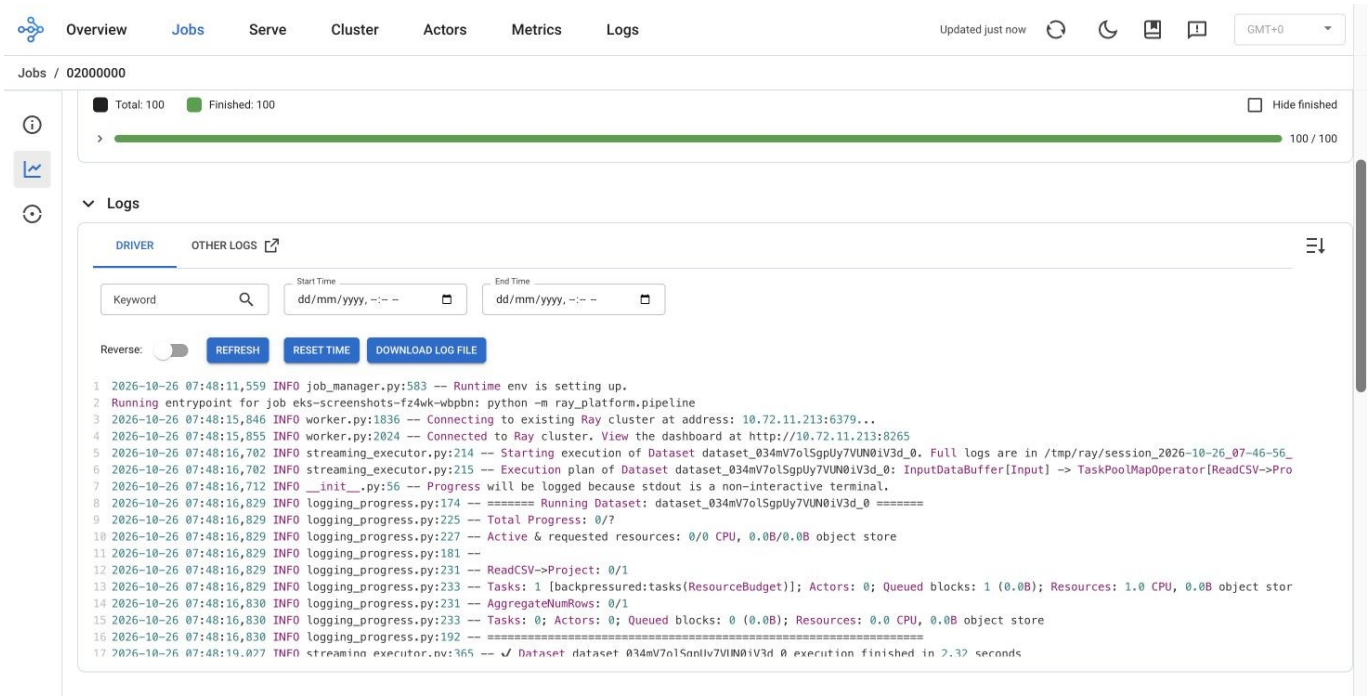
These four screenshots show the actual EKS environment on 26 October 2026, accessed through localhost tunnels. The separate local rehearsal remains in docs/evidence/rehearsal. Full-resolution cloud captures are in docs/evidence/eks-rehearsal.



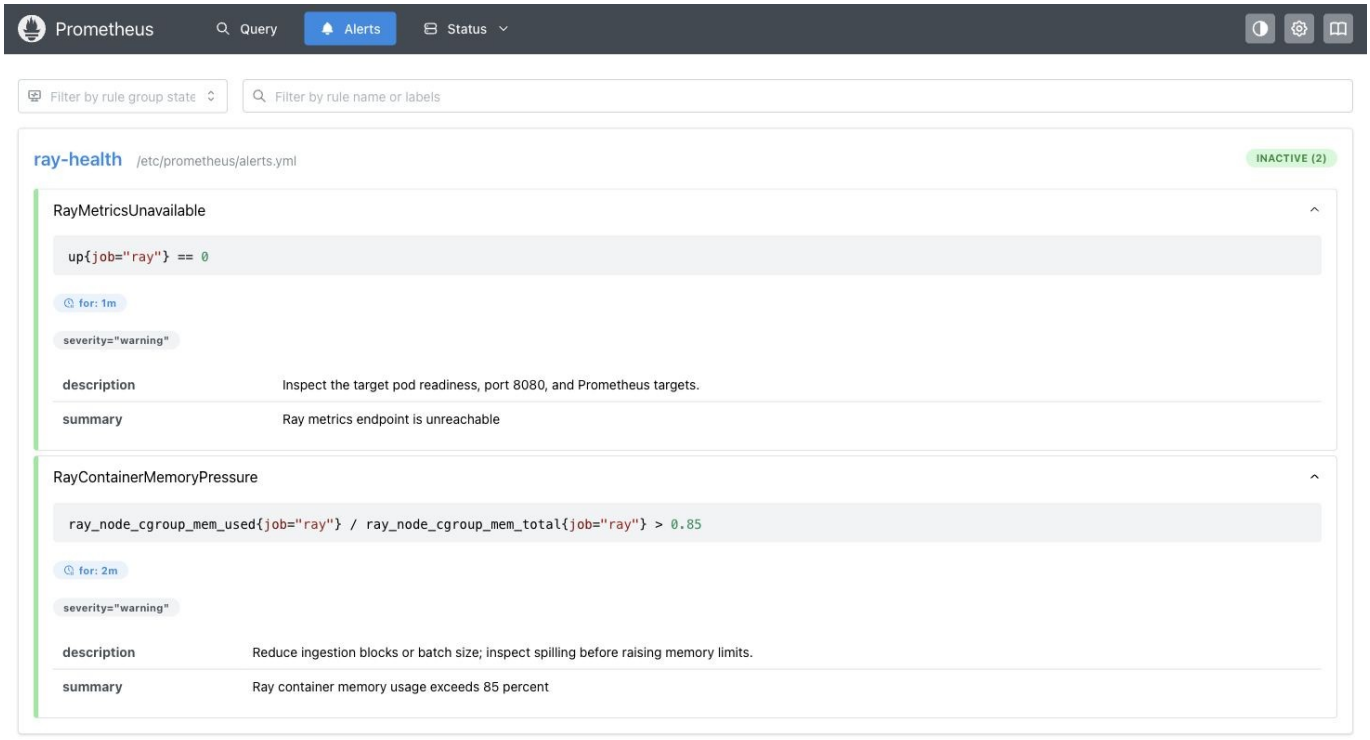
Two scrape targets and one retained Ray cluster. Grafana uses Asia/Kolkata time. The gaps show worker turnover during the retained demonstration window.



Submission eks-screenshots-fz4wk-wbpbn succeeded in 15.072 seconds. Ray timestamps are UTC. The retained cluster continued reporting task counters after completion.



The same submission appears in the driver logs. Long lines are clipped by the native viewer; this screenshot is not a full archive or a CloudWatch UI capture.



Both rules are inactive during normal operation. Saved promtool fixtures verify pending, firing and recovery. No Alertmanager or external notification delivery is claimed.

Known limitations and production work

The demonstrated implementation has explicit production gaps: single system node and NAT, bounded worker capacity, ephemeral Prometheus storage, short CloudWatch retention, no external paging, no public

domain/certificate and no HA monitoring. Localhost tunnels provide the demonstration access path. Ray's embedded Grafana panels are not wired; standalone Grafana works. The retained screenshot cluster showed repeated idle worker turnover and scrape gaps after job completion. Autoscaler interaction needs investigation and tuning before using long-lived Ray clusters in production.

Driver materialization is capped at 100000 rows. Larger inputs need partitioned Parquet, streaming transformations and distributed joins. Distributed Ray Train needs a simultaneous worker group, framework configuration and durable training checkpoints. The tested CPU baseline restarts training after failure. Entity resolution assumes valid registration identifiers and one record per company per source. Multi-writer conflict handling, GPU workloads and real LLM integration are outside the demonstrated fixture.

The newer local image candidate has 50 HIGH findings in the recorded OS/Python scan, excludes Java archives, and is not golden eligible. It was not pushed to ECR. Historic release success is not proof that a current vulnerable image passes the added gate. Remediation and a complete approved scan are required before a production release. The narrowly scoped HTTPS egress exception is temporary and expires on 1 November 2026; it is not domain-based outbound control.

The total AWS ceiling is USD 30 before credits. Posted gross spend was USD 4.18 at the post-cleanup check on 26 October; usage is delayed and this is not the final bill. KMS and retained storage can continue to incur cost after compute cleanup. Budget alerts and lease checks supplement manual verification.

Proposed production availability

The production objective is disaster resilience with verified recovery. No system is disaster proof. AWS manages the EKS control plane across Availability Zones; we remain responsible for resilient capacity, application state and dependencies. The following design is proposed only and is outside the USD 30 demonstration.

Failure boundary	Proposed production control	Acceptance evidence
Availability Zone	Private subnets across three Singapore AZs; On-Demand system capacity in each AZ, topology spread for replicated services and enough surviving capacity for critical jobs. Use per-AZ egress or redundant private endpoints and controlled proxies.	Remove one AZ from service under representative load; prove critical jobs meet their deadlines and private AWS access still works.
Spot capacity	Diversify worker instance families and AZs; bounded task retries and durable checkpoints; allow capped On-Demand fallback for time-critical work. Set resource quotas, requests and autoscaling limits.	Interrupt workers and simulate unavailable Spot capacity; verify recovery, no duplicate commits and bounded cost.
Ray head or driver	Keep one logical head per Ray cluster. An external scheduler and durable job ledger detect stalled execution, fence the old attempt and submit a replacement from a committed checkpoint or replayable input.	Kill the head during each pipeline stage; demonstrate recovery and one committed output per input batch.
Cluster services	Spread CoreDNS and other replicated services; use disruption budgets for voluntary maintenance. Enable KubeRay operator replicas only with supported leader election in the pinned version.	Drain nodes, upgrade the operator and restore controllers without losing scheduling ownership.

A disruption budget does not prevent AZ loss or Spot termination. Multiple Ray heads are not interchangeable replicas. Ray's published GCS guidance limits supported Redis-backed recovery to KubeRay Ray Serve; it does not establish transparent recovery of this batch driver. Evaluate any GCS persistence separately against

pinned versions. The batch recovery contract remains external job replay, durable state and idempotent commits. Current training restarts from the beginning; durable training checkpoints require application changes before claiming resume.

Proposed disaster recovery objectives

Use a pilot-light secondary region for this batch workload: continuously protect data, images and recovery configuration, then provision compute during recovery. A candidate is Sydney, subject to client data residency approval, service support and quota verification. A warm standby is an alternative if measured provisioning time cannot meet the agreed recovery target. Neither option is implemented.

RTO is elapsed time from service disruption to validated processing restoration, including detection and provisioning. RPO is the age of the latest consistent, recoverable data checkpoint. These starting targets require business approval and a timed recovery exercise; they are not an SLA or measured results.

Scenario	Proposed target and recovery unit	Qualification
Worker or single AZ loss	Restore critical processing within 30 minutes; replay uncommitted work from durable input.	No loss of acknowledged commits is a design goal, conditional on intact regional storage and catalog. Spare capacity and workload size must be tested.
Regional outage	RTO four hours; RPO one hour for a verified data, catalog and job-ledger recovery checkpoint.	Asynchronous replication alone does not guarantee this RPO. Monitor checkpoint age; escalate any breach before declaring recovery ready.
Corruption or accidental deletion	RTO eight hours; restore a verified clean daily recovery point, with an initial RPO target of 24 hours.	Detection delay may require an older backup and greater data loss. Retention and replayable input must cover that risk.

Protect S3 inputs, Iceberg files, models and checkpoints using versioning and cross-region replication, with destination KMS keys and tested restore-role access. Backfill pre-existing objects separately. Maintain independently retained recovery points in a backup account with approved immutability and retention policies; replication alone is not protection against every logical corruption scenario.

Restore Glue catalog definitions together with their matching Iceberg metadata, manifests and data files. A recovery checkpoint must record snapshot IDs, metadata locations, required object versions, model provenance and job-ledger position. Iceberg file references may point to the primary bucket: validate a supported relocation or metadata rewrite process so secondary queries do not depend on the failed region. Merely copying S3 objects is not a completed database recovery.

Replicate approved ECR releases and backfill required existing image digests; verify signatures and SBOM availability at the destination. Back up Terraform state and bootstrap configuration with separate recovery access. Keep state locking and one active infrastructure writer; do not apply from an arbitrary stale state copy. Recreate regional IRSA trust, secrets, KMS permissions and service configuration from reviewed code. Test account-compromise recovery with a separate break-glass identity and audit trail, independent of the primary cluster.

Regional recovery runbook proposal

1. The on-call engineer starts the incident clock, identifies the failure boundary and informs the incident commander. The commander authorizes regional recovery when the service target is at risk. Disable primary scheduling and fence its write permissions; prove exclusive write ownership before enabling the secondary.

2. The data owner selects the latest verified consistent recovery checkpoint. Check replication completeness, backup integrity, decryption permissions and recoverable catalog references. Record the actual potential data loss. If the RPO is exceeded, obtain a business decision rather than silently claiming compliance.

3. The platform engineer provisions the secondary stack from pinned Terraform, images and Helm versions. Restore catalog, secrets and job ledger; verify DNS, network access, OIDC/IRSA, quotas and monitoring. Reconcile infrastructure state before any apply. Primary-region services must not be hidden dependencies.

4. Run a canary that reads recovered inputs, queries an Iceberg snapshot and writes an isolated output. Verify row counts, checksums, identity boundaries and model provenance. Resume the scheduler with stable batch identifiers and bounded replay; confirm no duplicate commits. Record observed RTO and RPO before closing recovery.

5. Fail back in a planned change window: stop new writes, reconcile snapshots and ledger, validate replication direction and transfer exclusive writer ownership. Keep rollback checkpoints, produce a post-incident report and update the runbook. Run quarterly recovery exercises and repeat after material platform or schema changes.

Production observability and operating model

Monitor the business outcome as well as infrastructure. Agree a batch completion window and data freshness objective with the client; measure eligible batches completed correctly within that window divided by all eligible scheduled batches. An initial 99.5 percent monthly objective is a discussion point, not a commitment. Use an error budget to decide when reliability work takes priority over releases.

Add alerts for missed schedules, queue age, failed or stalled jobs, data-quality failures, checkpoint age, replication lag, backup failures, IRSA errors, unschedulable pods and unexpected worker churn. Detect missing exporters as well as unhealthy ones: an up-equals-zero rule alone cannot cover a target removed from discovery. Correlate batch ID, Ray submission ID, image digest and Iceberg snapshot in logs.

For a team with limited monitoring operations capacity, evaluate Amazon Managed Service for Prometheus and Amazon Managed Grafana with organizational SSO. An alternative is multi-AZ Prometheus with durable remote storage and replicated Grafana backed by a supported shared database. In both cases, keep alert delivery and an external heartbeat independent of the workload cluster. Route actionable alerts to an on-call service with an owner, severity, runbook and escalation. Choose log retention, redaction and metrics cardinality limits against audit needs and cost. Trace cross-service operations where it helps diagnose failed batches.

Keep EKS and Ray administration private. Production dashboard access should use VPN or an approved access proxy, SSO and least privilege, with private DNS and a trusted certificate when serving HTTPS. A public domain or public ALB is not a prerequisite. The demonstration's authenticated localhost tunnels are adequate for its evidence session; they do not provide shared production access or HA.

Delivery controls and production acceptance

Workstream	Required production change	Accountable owner and gate
Release security	Remediate the recorded image findings, scan all relevant dependencies, publish an immutable signed ECR release with SBOM and provenance, and verify admission policy. Replace the expiring egress exception with tested controlled outbound access.	Security and platform owners approve scan results, exceptions and negative access tests.
Change control	Restore required independent review, CODEOWNERS and passing CI before merge. Protect production environments and OIDC roles; separate plan from approved apply. Pin module releases and test upgrades and rollback in staging.	Platform owner demonstrates an unauthorized change is blocked. Select repository visibility and subscription that support required enforcement.
Isolation and recovery	Separate production, nonproduction and backup/security accounts. Add default-deny network policy with tested service exceptions, secret rotation, audit logging and recovery-role access.	Security owner signs off boundaries; platform and data owners complete a timed restore exercise.
Application reliability	Implement durable job ownership, replay safety and checkpoints; remove driver scaling bottlenecks; investigate observed worker turnover and test representative data volumes.	Data and ML owners verify output consistency, failure recovery and capacity limits.
Service handoff	Document support hours, incident escalation, maintenance and patch cadence, dependency owners, restore procedures, quotas and cost allocation.	Client service owner accepts measured recovery results, operating costs and remaining risks.

Deliver this in stages: first close release-security and replay-safety blockers; then prove multi-AZ operation and alert delivery in staging; then validate regional recovery; finally obtain a production go-live decision with measured evidence. Rollback means restoring a known-good image/configuration and a compatible data checkpoint where required, not blindly reversing an Iceberg schema change.

The USD 30 ceiling applies to the completed demonstration. Production requires a separately approved forecast covering three-AZ baseline capacity, egress/endpoints, monitoring, backup retention, cross-region transfer, KMS and any standby control plane. Estimate using regional pricing and representative load, and track cost per successful batch. No HA or DR resources were created for this proposal.

Production design references

AWS EKS reliability and shared responsibility: https://docs.aws.amazon.com/eks/latest/best-practices/reliability.html

Ray GCS recovery scope and limitations: https://docs.ray.io/en/latest/ray-core/fault_tolerance/gcs.html

S3 replication behavior and existing-object considerations: https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication-what-is-isnot-replicated.html

References reviewed 26 October 2026. Validate behavior against pinned release versions before implementation. The specific architecture, recovery targets and operating procedures above are recommendations for client review.

Managed alternative

EventBridge Scheduler → Step Functions → Glue Spark ingestion and Iceberg updates → an ECS Fargate task for CPU sklearn training → S3 model artifacts removes EKS and node/operator management. Use scoped roles per stage and bounded retries. This requires adapting the Ray APIs and is a documented alternative, not a deployed or measured cheaper solution. Larger distributed/GPU training could use an on-demand SageMaker training job instead of the Fargate stage.

Validation record and handoff

At the module review checkpoint, 49 pytest tests passed. On evidence-delivery commit 545e028, PR checks validate, security and pr-title passed; cloud-plan was intentionally skipped. The production proposal is documentation only and does not extend the tested cloud capabilities. Historic successful AWS runs include main/PR planning and merged release execution; links are in docs/cicd-validation.md. The submission must include this review branch, the final document and sanitized evidence; it should not imply PR merge or a production deployment has occurred.